

Generative AI with Diffusion Models

Vincenzo Pomponi

1 Introduction

In the previous lecture, we established the mathematical foundations of diffusion models. In particular, we introduced the forward diffusion process, which gradually corrupts data by adding noise over multiple steps, and the reverse diffusion process, which aims to reconstruct structured data starting from noise.

We then formalized the learning problem by deriving the training objective. Starting from the intractable data likelihood, we introduced the Evidence Lower Bound (ELBO) and showed how it leads to a tractable optimization problem. Finally, we obtained a practical training loss based on noise prediction, which enables efficient learning of the reverse process.

In this lecture, we move from theory to practice.

The central question we address is the following:

How can we effectively generate samples from a trained diffusion model?

To answer this, we first study different **sampling procedures** for diffusion models. In particular, we focus on two fundamental approaches:

- **Denoising Diffusion Probabilistic Models (DDPM)**, which follow the original stochastic reverse process derived from the probabilistic formulation;
- **Denoising Diffusion Implicit Models (DDIM)**, which define an alternative (deterministic or partially deterministic) reverse process that enables faster sampling.

Although both methods rely on the same trained model, they differ in how the reverse trajectory is constructed during generation, leading to different trade-offs between sample quality, diversity, and computational efficiency.

In the second part of this lecture, we turn to the **architectural design** of diffusion models.

We will analyze how neural networks are used to parameterize the reverse process, with particular emphasis on:

- the use of U-Net architectures,
- the incorporation of time-step information,
- and the role of noise conditioning in guiding the denoising process.

By the end of this lecture, you will have a clear understanding of how diffusion models are implemented in practice, how different sampling strategies affect generation, and how architectural choices influence the performance of these models.

2 DDPM: Denoising Diffusion Probabilistic Model

After training, the diffusion model has learned a neural network

$$\epsilon_{\theta}(\mathbf{x}_t, t),$$

whose goal is to estimate the noise contained in a noisy sample $\mathbf{x}_t$ at timestep t .

The reverse diffusion process uses this learned noise estimator to progressively transform pure Gaussian noise into a structured data sample. Sampling therefore consists of iteratively applying the learned reverse transitions:

$$\mathbf{x}_T \rightarrow \mathbf{x}_{T-1} \rightarrow \dots \rightarrow \mathbf{x}_0.$$

The process starts from:

$$\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I}),$$

which corresponds to pure isotropic Gaussian noise.

At each timestep t , the model receives the current noisy sample $\mathbf{x}_t$ and predicts the noise component:

$$\epsilon_{\theta}(\mathbf{x}_t, t).$$

The key idea is that, if the model can accurately estimate the noise contained in $\mathbf{x}_t$, then it can also infer how to move toward a cleaner sample. More precisely,

if we can estimate the noise component contained in $\mathbf{x}_t$, then we can subtract it and recover a cleaner version of the sample $\mathbf{x}_0$.

Recall the forward diffusion process:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}).$$

This equation shows that $\mathbf{x}_t$ is obtained by combining:

- a scaled clean signal $\mathbf{x}_0$,
- Gaussian noise ϵ .

During the sampling procedure, the original clean sample $\mathbf{x}_0$ is not available. Consequently, the neural network must infer the underlying noise structure directly from the current noisy observation. For this reason, the model takes as input:

$$(\mathbf{x}_t, t)$$

as input and predicts:

$$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \approx \boldsymbol{\epsilon}.$$

If the prediction is accurate, we can isolate the clean signal from Eq. (16). Starting from:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon},$$

subtract the estimated noise term:

$$\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \approx \sqrt{\bar{\alpha}_t} \mathbf{x}_0.$$

Dividing by $\sqrt{\bar{\alpha}_t}$ gives:

$$\hat{\mathbf{x}}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)). \quad (1)$$

Therefore, predicting the noise is equivalent to estimating the original clean sample. Deriving this expression is important because it establishes the connection between *noise prediction* and *data reconstruction* in diffusion models. In particular, it shows that, if the neural network can accurately estimate the noise contained in a noisy sample $\mathbf{x}_t$, then the original clean sample $\mathbf{x}_0$ can be directly recovered from it. This provides the theoretical justification for training diffusion models to predict noise instead of directly predicting the clean image itself.

Although $\hat{\mathbf{x}}_0$ is an estimate of the clean sample, directly using it as the final generated output at every timestep would be problematic.

First, the prediction $\hat{\mathbf{x}}_0$ is only an approximation obtained from a noisy observation $\mathbf{x}_t$. Especially for large timesteps t , the sample $\mathbf{x}_t$ contains a significant amount of noise, and therefore the estimate of $\mathbf{x}_0$ can still be inaccurate.

Moreover, diffusion models are designed to learn a *progressive denoising process*. Instead of reconstructing the final sample in a single step, the model performs many small denoising updates. This iterative refinement is much more stable and easier to learn than directly predicting the complete clean sample from highly corrupted noise.

For these reasons, the estimate $\hat{\mathbf{x}}_0$ is not used directly as the generated sample.

Instead of directly using the estimate $\hat{\mathbf{x}}_0$ as the final generated sample, diffusion models use it to construct the reverse transition distribution:

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t),$$

which generates a slightly less noisy sample $\mathbf{x}_{t-1}$ from the current noisy state $\mathbf{x}_t$.

The reverse generation process is therefore iterative, where each step progressively removes a small amount of noise.

The true reverse posterior of the reverse process is:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0),$$

which is Gaussian and has analytical mean:

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t} \mathbf{x}_0. \quad (2)$$

However, during generation the clean sample $\mathbf{x}_0$ is unknown. Therefore, we substitute $\mathbf{x}_0$ with its estimate $\hat{\mathbf{x}}_0$ obtained from the predicted noise. Substituting Eq. (1) into Eq. (2), after algebraic simplifications, yields the parameterized reverse mean:

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right). \quad (3)$$

The quantity

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$$

should not be interpreted as the final reconstructed sample. Instead, it represents the center (mean) of the Gaussian distribution used to generate the next denoising step.

The reverse transition is therefore modeled as:

$$p(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N} \left(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \tilde{\beta}_t \mathbf{I} \right), \quad (4)$$

where:

- $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ determines the denoising direction,
- $\tilde{\beta}_t \mathbf{I}$ controls the stochasticity of the reverse process.

Sampling from this Gaussian gives:

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_\theta(\mathbf{x}_t, t) + \sqrt{\tilde{\beta}_t} \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, \mathbf{I}). \quad (5)$$

Equation (5) reveals the two fundamental components of DDPM sampling:

Deterministic denoising component

The term $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ defines the direction toward a cleaner sample. It is entirely determined by the neural network prediction $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$. Therefore, the denoising direction is deterministic once the network output is fixed.

Stochastic sampling component

The term $\sqrt{\tilde{\beta}_t}\mathbf{z}$, with $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$, introduces randomness into the reverse process.

This stochasticity originates from the probabilistic formulation of diffusion models. Indeed, the reverse process is modeled as a Gaussian distribution rather than as a deterministic mapping. Consequently, even if two trajectories start from the same noisy sample, different Gaussian noise realizations may produce different outputs.

The variance parameter $\tilde{\beta}_t$ controls the amount of randomness injected at each reverse step:

- small variance leads to more stable denoising,
- larger variance increases diversity in generated samples.

Summary

The complete DDPM sampling algorithm can be summarized as follows:

1. Sample initial Gaussian noise:

$$\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I}).$$

2. For $t = T, \dots, 1$:

- (a) Predict the noise:

$$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t).$$

- (b) Compute the reverse mean:

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right).$$

- (c) Sample the next state:

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_\theta(\mathbf{x}_t, t) + \sqrt{\tilde{\beta}_t} \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, \mathbf{I}).$$

3. Return the final generated sample $\mathbf{x}_0$.

Thus, DDPM sampling alternates between:

1. a deterministic denoising operation driven by the neural network,
2. a stochastic sampling operation induced by Gaussian noise injection.

This stochastic formulation provides a principled probabilistic interpretation of generation and allows diffusion models to produce diverse and high-quality samples.

Deterministic Neural Network Prediction

In DDPMs, the neural network $\epsilon_\theta(\mathbf{x}_t, t)$ is typically a deterministic function once:

- the parameters θ are fixed,
- the input $\mathbf{x}_t$ is fixed,
- the timestep t is fixed.

That means:

$$(\mathbf{x}_t, t) \rightarrow \epsilon_\theta(\mathbf{x}_t, t)$$

always produces the same output.

For example, if you feed exactly the same noisy image $\mathbf{x}_t$ into the network twice, the predicted noise will be identical (assuming inference mode and no dropout/random layers).

3 DDIM: Denoising Diffusion Implicit Models

Denoising Diffusion Implicit Models (DDIMs) were introduced to accelerate the sampling procedure of diffusion models while preserving the same training objective used in DDPMs.

The key idea behind DDIM is that the reverse diffusion process does not necessarily need to be fully stochastic. In DDPMs, each reverse step injects additional Gaussian noise:

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_\theta(\mathbf{x}_t, t) + \sqrt{\tilde{\beta}_t} \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, \mathbf{I}),$$

which makes the trajectory random.

DDIM instead constructs a *non-Markovian* reverse process that preserves the same marginal distributions

$$q(\mathbf{x}_t | \mathbf{x}_0),$$

while allowing deterministic sampling trajectories. In other words, although DDIM modifies the reverse dynamics and can even make them deterministic, it preserves the statistical relationship between $\mathbf{x}_0$ and $\mathbf{x}_t$ defined by the original forward process.

The starting point is the forward diffusion equation:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

As in DDPM, the neural network predicts the noise:

$$\epsilon_\theta(\mathbf{x}_t, t).$$

Using this prediction, we can estimate the original clean sample $\mathbf{x}_0$ by rearranging the forward diffusion equation:

$$\hat{\mathbf{x}}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right). \quad (6)$$

This equation represents an estimate of the original clean data point before noise was added.

- $\mathbf{x}_t$ is the current noisy sample,
- $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ is the predicted noise,
- subtracting the estimated noise removes the corruption,
- dividing by $\sqrt{\bar{\alpha}_t}$ rescales the signal back to the original data domain.

Therefore, DDIM explicitly reconstructs an estimate of the clean image at every reverse step.

Once $\hat{\mathbf{x}}_0$ is estimated, DDIM defines the reverse update as (see Appendix A for more details):

$$\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{\mathbf{x}}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) + \sigma_t \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, \mathbf{I}). \quad (7)$$

This expression can be interpreted as the combination of three terms:

1.
$$\sqrt{\bar{\alpha}_{t-1}} \hat{\mathbf{x}}_0$$
 preserves the reconstructed clean signal,
2.
$$\sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$$
 controls the direction of the denoising trajectory using the predicted noise,
3.
$$\sigma_t \mathbf{z}$$
 optionally injects additional stochasticity.

Unlike DDPM, the amount of randomness is now explicitly controlled by the parameter σ_t .

Deterministic DDIM

The most important special case is obtained by setting:

$$\sigma_t = 0.$$

In this case, the reverse process becomes completely deterministic:

$$\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\hat{\mathbf{x}}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t). \quad (8)$$

No Gaussian noise is sampled during the reverse step. Consequently, the entire generation trajectory is uniquely determined by:

- the initial noise sample $\mathbf{x}_T$,
- the neural network predictions.

Therefore:

$$\text{same initial noise} \implies \text{same generated sample.}$$

4 DDPM vs DDIM

The key distinction between DDPM and DDIM lies in the reverse sampling process:

- DDPM performs stochastic sampling by injecting Gaussian noise at every step,
- DDIM can perform deterministic sampling by removing the stochastic term.

As a consequence:

- DDPM better approximates the probabilistic reverse diffusion process,
- DDIM produces faster sampling trajectories and often requires fewer denoising steps.

An important observation is that DDIM uses the *same trained network* as DDPM. Only the sampling procedure changes.

Therefore, DDIM can be interpreted as an alternative inference procedure for diffusion models, designed to accelerate generation while preserving sample quality.

5 Architectures for Diffusion Models

Diffusion models require a neural network capable of estimating the noise contained in a noisy sample:

$$\epsilon_{\theta}(\mathbf{x}_t, t).$$

The architecture receives as input:

- a noisy sample $\mathbf{x}_t$,
- the diffusion timestep t ,

and predicts the noise that has been added during the forward diffusion process. The predicted noise is then used to reconstruct cleaner samples during the reverse diffusion procedure.

Different architectures can be used depending on the structure of the data:

- **UNet-2D**: images,
- **UNet-1D**: temporal sequences and trajectories,
- **Transformer-based architectures (DiT)**: patch/token-based representations.

5.1 Why U-Net Architectures?

The U-Net architecture is particularly suitable for diffusion models because denoising requires:

1. extracting global semantic information,
2. preserving fine local details,
3. reconstructing high-resolution outputs.

The U-Net achieves this through:

- an **encoder** path that progressively compresses the representation,
- a **decoder** path that reconstructs the signal,
- **skip connections** that preserve spatial details.

The general structure is:

$$\text{Input} \rightarrow \text{Encoder} \rightarrow \text{Bottleneck} \rightarrow \text{Decoder} \rightarrow \text{Output}.$$

In diffusion models, the output has the same dimensionality as the input because the network predicts the noise associated with each input component.

5.2 Time Conditioning

A diffusion model must know *how much noise* is contained in the current sample $\mathbf{x}_t$.

For this reason, the timestep t is embedded into a high-dimensional representation using sinusoidal embeddings:

$$\gamma(t) = [\sin(\omega_1 t), \cos(\omega_1 t), \dots, \sin(\omega_k t), \cos(\omega_k t)].$$

This embedding is then processed through a small MLP:

$$\mathbf{e}_t = \text{MLP}(\gamma(t)).$$

The resulting vector conditions the entire network and informs it about the current diffusion step.

In the provided implementation, this is performed by:

- SinusoidalEmbedding,
- TimeEmbedMLP.

5.3 UNet-2D

What Does “2D” Mean? The term *2D* refers to the fact that the neural network operates on data defined over two spatial dimensions. In the context of diffusion models, this usually corresponds to images, which are naturally represented on a two-dimensional grid of pixels.

A UNet-2D therefore processes tensors of shape:

$$(B, C, H, W),$$

where:

- B is the batch size,
- C is the number of channels,
- H is the image height,
- W is the image width.

The two dimensions (H, W) define the spatial structure of the image:

- H corresponds to the vertical axis,
- W corresponds to the horizontal axis.

For example, the tensor:

$$(2, 1, 28, 28)$$

represents:

- a batch containing 2 samples,
- each sample having 1 channel (grayscale),
- image resolution equal to 28×28 pixels.

Similarly:

$$(16, 3, 256, 256)$$

would represent:

- a batch of 16 RGB images,
- each image having 3 channels,
- spatial resolution 256×256 .

2D Convolutions The architecture uses `Conv2d` layers:

$$\text{nn.Conv2d}(C_{\text{in}}, C_{\text{out}}, k), \quad (9)$$

which apply convolutional filters over both spatial dimensions simultaneously. A convolution kernel slides across the image and extracts local spatial patterns such as:

- edges,
- corners,
- textures,
- shapes,
- semantic structures.

For instance, a 3×3 convolution examines a local neighborhood of pixels:

$$\begin{bmatrix} x_{i-1,j-1} & x_{i-1,j} & x_{i-1,j+1} \\ x_{i,j-1} & x_{i,j} & x_{i,j+1} \\ x_{i+1,j-1} & x_{i+1,j} & x_{i+1,j+1} \end{bmatrix}.$$

The same kernel weights are reused across the entire image. This property is known as *weight sharing* and allows the network to efficiently detect similar patterns at different spatial locations.

Why UNet Works Well for Diffusion Models In diffusion models, the network receives a noisy image:

$$\mathbf{x}_t,$$

and predicts the noise:

$$\epsilon_{\theta}(\mathbf{x}_t, t).$$

To accurately estimate the noise, the model must simultaneously:

- preserve fine local details,
- capture large-scale global structures.

The UNet architecture is particularly effective because it combines:

- an *encoder*, which progressively compresses the image and extracts high-level semantic information,
- a *decoder*, which reconstructs spatial details,
- *skip connections*, which transfer high-resolution information directly from encoder to decoder.

This enables the model to denoise images while preserving both:

- global coherence,
- local image quality.

5.4 Encoder

The encoder is the *contracting path* of the U-Net architecture. Its purpose is to progressively transform the input image into increasingly abstract feature representations.

Starting from the original image resolution:

$$28 \times 28,$$

the encoder repeatedly applies:

- convolutional layers,
- nonlinear activations,
- downsampling operations (pooling).

As a result, the spatial resolution decreases:

$$28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7.$$

Why Reduce Spatial Resolution? Reducing the spatial dimensions allows the network to:

- increase the receptive field,
- capture larger spatial dependencies,
- extract higher-level semantic information,
- reduce computational cost.

For example:

- early layers focus on local structures such as edges or textures,
- deeper layers capture larger patterns such as shapes, objects, or global image structure.

The downsampling is typically performed using pooling layers:

$$\text{MaxPool2d}(2),$$

which reduce each spatial dimension by a factor of 2:

$$(H, W) \rightarrow \left(\frac{H}{2}, \frac{W}{2} \right).$$

Thus:

$$28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7.$$

Increase in Feature Channels While the spatial resolution decreases, the number of feature channels increases:

$$64 \rightarrow 128 \rightarrow 256.$$

This means that:

- the representation becomes spatially smaller,
- but semantically richer.

Each channel can be interpreted as a learned feature detector. For instance:

- some channels may activate on edges,
- others on textures,
- others on object parts or global structures.

Increasing the number of channels allows the network to store more information despite the reduced spatial size.

Conceptually:

$$\text{smaller spatial size} \implies \text{more abstract representation.}$$

Trade-Off Between Resolution and Semantics The encoder performs a trade-off:

- spatial precision is progressively reduced,
- semantic understanding is progressively increased.

For example:

- the first layers preserve detailed pixel-level information,
- deeper layers encode more global and abstract image properties.

This hierarchical representation is essential for diffusion models because the network must understand:

- local image details,
- global image structure,
- long-range spatial correlations,

in order to accurately predict the noise contained in the noisy sample.

Encoder in Diffusion Models In diffusion models, the encoder receives a noisy image:

$$\mathbf{x}_t,$$

and progressively extracts meaningful features from it.

Even when the image is highly corrupted by Gaussian noise, the encoder learns to identify:

- underlying structures,
- spatial patterns,
- semantic content.

These latent representations are then used by the decoder to reconstruct a cleaner version of the image and estimate the noise:

$$\epsilon_{\theta}(\mathbf{x}_t, t).$$

Therefore, the encoder acts as a hierarchical feature extractor that transforms noisy low-level pixel information into high-level semantic representations useful for denoising.

5.5 Residual Blocks

The architecture uses residual blocks:

$$\mathbf{y} = F(\mathbf{x}) + \mathbf{x}.$$

Residual connections improve:

- gradient flow,
- optimization stability,
- training of deep architectures.

5.6 Self-Attention

The bottleneck includes self-attention layers.

Convolutions are local operations because they only observe nearby pixels. Self-attention allows the model to capture long-range dependencies:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V.$$

This is important because:

- distant image regions may be correlated,
- global consistency is required,
- semantic structures often extend over large spatial areas.

5.7 Decoder

The decoder progressively reconstructs the original resolution:

$$7 \times 7 \rightarrow 14 \times 14 \rightarrow 28 \times 28.$$

Skip connections concatenate encoder and decoder features:

$$\text{DecoderFeatures} = [\text{UpsampledFeatures}, \text{EncoderFeatures}].$$

This preserves:

- edges,
- textures,
- fine image details.

6 UNet-2D with Full Attention

The second architecture extends the previous model by inserting self-attention at every resolution level.

Instead of applying attention only at the bottleneck:

Attention only at lowest resolution,

attention is applied throughout the encoder and decoder.

This improves:

- global coherence,
- long-range interactions,
- semantic consistency.

However, attention is computationally expensive because its complexity scales quadratically:

$$\mathcal{O}(N^2),$$

where N is the number of spatial positions.

7 UNet-1D

7.1 What Does “1D” Mean?

UNet-1D processes signals defined along a *single dimension*.

Instead of images:

$$(H, W),$$

the data are sequences:

$$(T),$$

where T is the temporal horizon.

The input shape becomes:

$$(B, C, T).$$

For example:

$$(B, D_{\text{action}}, T_{\text{horizon}}).$$

In the provided implementation:

- D_{action} represents action dimensions,
- T_{horizon} represents the sequence length.

This architecture is useful for:

- trajectories,

- robotics,
- time series,
- motion generation,
- audio signals.

7.2 Difference Between UNet-1D and UNet-2D

The key difference is the dimensionality of the convolution.

UNet-2D:

Conv2d

processes:

(H, W) .

UNet-1D:

Conv1d

processes:

(T) .

Conceptually:

- UNet-2D learns spatial patterns,
- UNet-1D learns temporal patterns.

For example:

- 2D convolution learns image edges and textures,
- 1D convolution learns motion evolution and temporal dependencies.

8 Diffusion Transformers (DiT)

Recent diffusion models increasingly replace convolutions with Transformers. Instead of processing images through convolutions, the image is divided into patches:

Image $\rightarrow$ Patches $\rightarrow$ Tokens.

Each patch is embedded into a vector representation:

$$\mathbf{z}_i \in \mathbb{R}^d.$$

The Transformer then processes the sequence of patch tokens using self-attention.

8.1 Patchification

Given:

$$x \in \mathbb{R}^{B \times C \times H \times W},$$

the image is divided into patches of size:

$$P \times P.$$

The number of patches is:

$$N = \left(\frac{H}{P}\right) \left(\frac{W}{P}\right).$$

Each patch becomes a token.

8.2 Transformer Blocks

Each DiT block performs:

1. self-attention,
2. feed-forward transformation,
3. timestep conditioning.

Unlike U-Nets:

- there is no encoder-decoder hierarchy,
- there are no convolutions,
- the architecture is fully attention-based.

8.3 Advantages of DiT

Transformers provide:

- stronger global modeling,
- scalability,
- better long-range dependency learning.

This is particularly useful for:

- high-resolution image generation,
- video generation,
- multimodal diffusion models.

9 Summary

The architectures presented differ mainly in how they process the structure of the data.

Architecture	Data Type	Main Operation
UNet-2D	Images	2D convolutions
UNet-1D	Sequences	1D convolutions
DiT	Patch tokens	Self-attention

In all cases, the objective remains the same:

$$\epsilon_{\theta}(\mathbf{x}_t, t),$$

namely predicting the noise contained in the noisy sample $\mathbf{x}_t$ in order to reverse the diffusion process.

A DDIM Update Equation

The key idea behind DDIM is to design an alternative reverse diffusion process whose intermediate distributions remain consistent with the marginals of the original forward diffusion process. In particular, the forward process defines the distribution

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I}),$$

which specifies how a clean sample $\mathbf{x}_0$ is progressively corrupted by Gaussian noise at each timestep t .

DDIM constructs a reverse sampling procedure such that, at every timestep, the generated sample $\mathbf{x}_t$ still follows this same marginal distribution. In other words, although DDIM modifies the reverse dynamics and can even make them deterministic, it preserves the statistical relationship between $\mathbf{x}_0$ and $\mathbf{x}_t$ defined by the original forward process.

Formally, DDIM seeks a reverse process satisfying

$$q_{\text{DDIM}}(\mathbf{x}_t|\mathbf{x}_0) = q(\mathbf{x}_t|\mathbf{x}_0)$$

for all timesteps t . Therefore, the reverse trajectory can be changed without altering the overall marginal distributions induced by the diffusion process.

Starting from the forward diffusion process,

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}), \quad (10)$$

the neural network predicts the noise component

$$\epsilon_{\theta}(\mathbf{x}_t, t).$$

Using this prediction, an estimate of the clean sample can be reconstructed as

$$\hat{\mathbf{x}}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t}\epsilon_{\theta}(\mathbf{x}_t, t)). \quad (11)$$

The objective is then to generate a sample $\mathbf{x}_{t-1}$ whose marginal distribution matches

$$q(\mathbf{x}_{t-1}|\mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0, (1 - \bar{\alpha}_{t-1})\mathbf{I}).$$

To satisfy this constraint, DDIM decomposes $\mathbf{x}_{t-1}$ into a reconstructed signal term, a directional component aligned with the predicted noise, and an optional stochastic term:

$$\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\hat{\mathbf{x}}_0 + \mathbf{v} + \sigma_t\mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, \mathbf{I}), \quad (12)$$

where σ_t controls the amount of injected randomness. Since the target variance at timestep $t - 1$ is

$$1 - \bar{\alpha}_{t-1},$$

and the stochastic component contributes variance σ_t^2 , the remaining variance must be captured by the deterministic direction $\mathbf{v}$. DDIM chooses this direction to align with the predicted noise:

$$\mathbf{v} = \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t). \quad (13)$$

Substituting this expression into Eq. (12) yields the DDIM update equation:

$$\boxed{\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\hat{\mathbf{x}}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) + \sigma_t\mathbf{z}} \quad (14)$$

with

$$\mathbf{z} \sim \mathcal{N}(0, \mathbf{I}).$$

This formulation explicitly separates the reconstructed clean sample, the predicted noise direction, and the optional stochastic exploration term. The parameter σ_t therefore determines the degree of randomness in the reverse process. When $\sigma_t > 0$, sampling remains partially stochastic, whereas setting

$$\sigma_t = 0$$

produces the deterministic DDIM sampler:

$$\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\hat{\mathbf{x}}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t). \quad (15)$$

In this deterministic setting, no additional random noise is injected during generation.

B Explanation

In DDPM, the forward diffusion process gradually corrupts the original data sample $\mathbf{x}_0$ by adding Gaussian noise at each timestep. After many diffusion steps, the sample $\mathbf{x}_t$ becomes increasingly noisy and eventually approaches pure Gaussian noise.

The forward process can be written in closed form as:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}). \quad (16)$$

This equation shows that $\mathbf{x}_t$ is composed of two parts:

- a scaled version of the original signal $\mathbf{x}_0$,
- an additive Gaussian noise component $\boldsymbol{\epsilon}$.

The central idea of DDPM is the following:

If we can estimate the noise component contained in $\mathbf{x}_t$, then we can subtract it and recover a cleaner version of the sample.

More precisely, the neural network receives:

$$(\mathbf{x}_t, t)$$

as input and predicts:

$$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \approx \boldsymbol{\epsilon}.$$

If the prediction is accurate, we can isolate the clean signal from Eq. (16).

Starting from:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon},$$

subtract the estimated noise term:

$$\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \approx \sqrt{\bar{\alpha}_t} \mathbf{x}_0.$$

Dividing by $\sqrt{\bar{\alpha}_t}$ gives:

$$\hat{\mathbf{x}}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)). \quad (17)$$

Therefore, predicting the noise is equivalent to estimating the original clean sample.

This explains why noise prediction is sufficient for learning the reverse diffusion process:

- the model identifies which part of $\mathbf{x}_t$ is noise,
- subtracting this noise moves the sample toward the data manifold,
- repeating this process iteratively produces progressively cleaner samples.

Intuitively, the neural network learns the direction that transforms noisy samples into realistic data samples.

References